

NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

DEPARTAMENTO DE ENGENHARIA ELETROTÉCNICA E
DE COMPUTADORES

Mestrado em Engenharia Eletrotécnica e de Computadores

Laboratório 1

Co-design e Sistemas Reconfiguráveis

Alunos:

João Pedro Antunes - **70380**

Júlio Lopes- **70512**

Marcos Romão - **71348**

Docentes:

Aniko Costa

Filipe Moutinho

1º Semestre 2025/2026

Conteúdo

1	Introdução	1
1.1	Contextualização	1
1.2	Objetivos	1
2	Análise Teórica	2
2.1	Redes de Petri	2
2.2	Execução síncrona vs. GALS	2
2.3	Implementação em FPGA e Arduino	3
3	Problema I - modelação de um sistema em IOPT-Tools	4
3.1	Rede de Petri — Tapete Singular	4
3.2	Rede de Petri — Modelo Completo	5
3.3	Rede de Petri — Fusion set	6
3.3.1	Rede de Petri — Node set	7
4	Problema II - Análise do modelo criado	7
4.1	Simulação com token-player	7
4.1.1	Geração e análise do espaço de estados	8
5	Problema III - Implementação em FPGA	9
5.1	Geração do código VHDL	9
5.2	Configuração de entradas/saídas	10
5.3	Simulações no Vivado	11
5.4	Testes Experimentais	11
6	Problema IV - Implementação em Arduino	12
6.1	Implementação no arduino e Geração do código C	12
6.2	Configuração de entradas/saídas	13
6.3	Testes Experimentais	14
7	Problema V - Controlador Distribuído em regime Síncrono	15
7.1	Cutting Set	15
7.2	Decomposição SPLIT	15
7.3	Modelo com canais síncronos	16
7.4	Modelo com canais assíncronos	16
7.5	Decomposição GALS	17
7.6	Simulação e Análise	17
7.6.1	Simulação com token-player	17
8	Problema VI - Implementação em FPGA com distribuição Síncrona	18
8.1	Geração do código VHDL	18
8.2	Simulação no Vivado	18
8.3	Testes Experimentais	19

9	Problema VII - Implementação em FPGA com distribuição Síncrona com atrasos entre eventos	20
9.1	Implementação do LFSR	20
9.1.1	Gestão dos atrasos	21
9.2	Testes Experimentais	21
10	Problema VIII - Buffers de Capacidade Finita	22
10.1	Modificação do modelo	22
10.2	Interconexão dos componentes	22
10.2.1	Geração e análise do espaço de estados	23
10.3	Simulação e Análise	24
10.3.1	Simulação com token-player	24
10.4	Implementação em FPGA	24
10.5	Testes experimentais	25
11	Problema IX - Distribuído com Buffers (Síncrono)	26
11.1	Aplicação síncrona ao buffer de capacidade finita	26
11.2	Decomposição e implementação distribuída	27
11.3	Simulação no Vivado	27
11.4	Testes Experimentais	28
12	Problema X - Distribuído com Buffers (Síncrono) e atraso entre eventos	29
12.1	Simulação no Vivado	29
12.2	Testes Experimentais	29
13	Comparação de abordagens	30
13.1	Centralizado vs Distribuído	30
13.2	Síncrono vs Assíncrono	30
13.3	Impacto dos atrasos de comunicação	30
14	Conclusões	31

1 Introdução

1.1 Contextualização

Este trabalho insere-se no âmbito da unidade curricular de Co-design e Sistemas Reconfiguráveis, focando-se no desenvolvimento e implementação de controladores para sistemas de automação industrial. O caso de estudo considerado é um sistema de produção composto por 3 células em cascata, onde cada célula integra um braço de robô e um tapete rolante.

A complexidade destes sistemas justifica a utilização de metodologias formais de modelação e verificação, permitindo validar o comportamento do controlador antes da sua implementação física. Adicionalmente, a possibilidade de implementar o controlador de forma centralizada ou distribuída oferece diferentes trade-offs entre complexidade de implementação, desempenho e escalabilidade do sistema.

1.2 Objetivos

O presente trabalho tem como objetivo principal o desenvolvimento completo de um controlador para um sistema de automação com três células em cascata ($N=3$), desde a fase de modelação até à implementação física em hardware reconfigurável.

Os objetivos específicos incluem:

- Modelar o comportamento do sistema utilizando redes de Petri Input-Output Place-Transition (IOPT), explorando as capacidades de composição modular através de operações de adição e fusão de redes;
- Validar e analisar o modelo através de simulação e análise do espaço de estados, verificando propriedades comportamentais críticas do sistema;
- Implementar o controlador em plataformas de hardware distintas (FPGA e Arduino), avaliando a abordagem centralizada de controlo;
- Desenvolver uma arquitetura de controlo distribuído através da decomposição do modelo global, explorando paradigmas de execução síncrona e GALS (Globally Asynchronous Locally Synchronous);
- Analisar o impacto de comunicações não-instantâneas entre controladores distribuídos, introduzindo atrasos pseudo-aleatórios;
- Estender o modelo para suportar buffers de capacidade finita superior a uma unidade, aumentando a flexibilidade do sistema;
- Comparar as diferentes abordagens de implementação, avaliando vantagens, desvantagens e adequação a diferentes contextos aplicacionais.

2 Análise Teórica

2.1 Redes de Petri

As redes de Petri constituem uma ferramenta formal de modelação adequada para sistemas concorrentes e distribuídos. Uma rede de Petri é definida por uma quádrupla $PN = (P, T, F, M_0)$, onde P representa lugares, T transições, F arcos direcionados, e M_0 a marcação inicial. A dinâmica baseia-se no disparo de transições, que removem tokens dos lugares de entrada e adicionam aos de saída.

Redes IOPT (Input-Output Place-Transition) estendem as redes clássicas com capacidade de interagir com o ambiente através de:

- Eventos de entrada associados a sinais externos (sensores);
- Sinais de saída para controlo de atuadores;
- Guardas e prioridades para resolução de conflitos;
- Semântica temporal para modelar delays.

Redes de Petri Coloridas permitem representar de forma compacta sistemas com estrutura repetitiva. Os tokens possuem "cores" (tipos de dados), permitindo que um único modelo represente múltiplas instâncias de subsistemas idênticos. No sistema estudado, tokens $\langle 1 \rangle$, $\langle 2 \rangle$ e $\langle 3 \rangle$ identificam cada célula.

Composição Modular: A construção de sistemas complexos utiliza operações de:

- *Merge Nets*: união de múltiplas redes mantendo nós distintos;
- *Fusion Sets*: identificação de nós a fundir criando sincronização;
- *Net Addition*: combinação de merge e fusão num modelo integrado.

2.2 Execução síncrona vs. GALS

Em contexto síncrono, todos os componentes operam com um relógio global comum. Oferece simplicidade de design e determinismo, mas apresenta limitações de escalabilidade, consumo energético elevado (relógio sempre ativo) e dificuldades na distribuição do sinal de relógio (clock skew) em sistemas grandes.

Já num paradigma GALS, este divide o sistema em domínios síncronos locais que comunicam assincronamente. Cada domínio opera com relógio independente, comunicando através de protocolos de handshaking ou FIFOs assíncronas.

Vantagens do GALS incluem escalabilidade, modularidade, eficiência energética e tolerância a falhas. As desvantagens são a maior complexidade de interface, latência variável na comunicação e necessidade de tratamento de metastabilidade nas fronteiras entre domínios.

2.3 Implementação em FPGA e Arduino

Utilizar-se-á uma **FPGA (Field-Programmable Gate Array)**. A descrição do sistema é feita em VHDL, uma linguagem de descrição de hardware que permite especificar comportamento concorrente. Recorrer-se-á também a um microcontrolador **Arduino**.

A framework IOPT-Tools permite geração automática de código VHDL para FPGA e código C para Arduino, facilitando a comparação entre as duas abordagens de implementação.

Critério	FPGA	Arduino
Paralelismo	Hardware real	Software (sequencial)
Tempo de resposta	ns	ms

Tabela 1: Comparação entre plataformas FPGA e Arduino

3 Problema I - modelação de um sistema em IOPT-Tools

No problema I, foi pedido para modelar um sistema de tapetes transportadores com garras robóticas com recurso a redes de Petri, utilizando a ferramenta IOPT-Tools.

A rede de alto nível dada no enunciado consiste em:

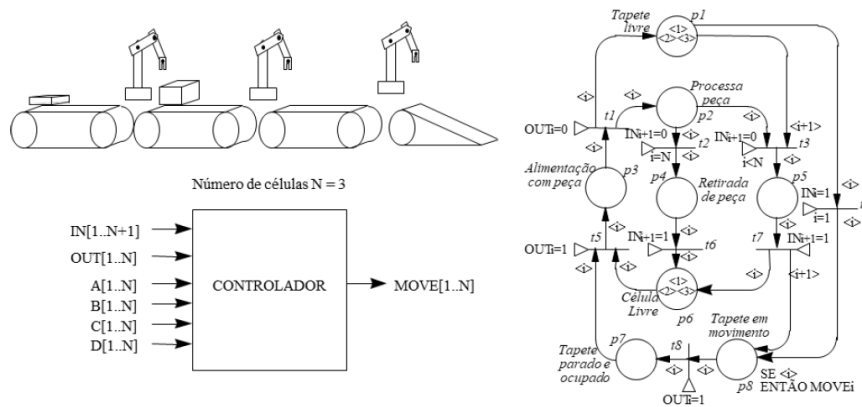


Figura 1: Rede de Petri — Alto Nível

Esta rede de alto nível permite implementar o controlador que vamos considerar daqui para a frente.

Na imagem 2, podemos observar a implementação de um conjunto tapete mais braço robótico.

3.1 Rede de Petri — Tapete Singular

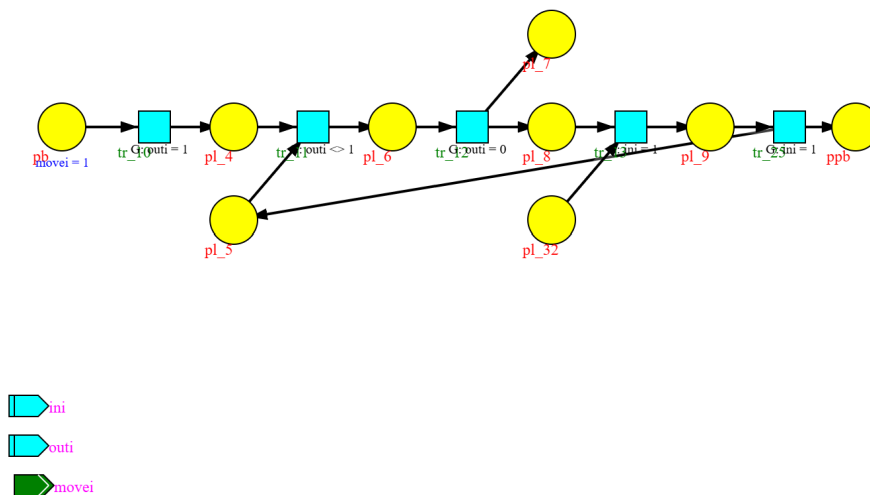


Figura 2: Rede de Petri — Tapete Singular

Atendendo á implementação da figura 2, podemos expandir o modelo para o sistema completo com 3 tapetes e 3 braços robóticos, como se pode observar na figura 3.

3.2 Rede de Petri — Modelo Completo

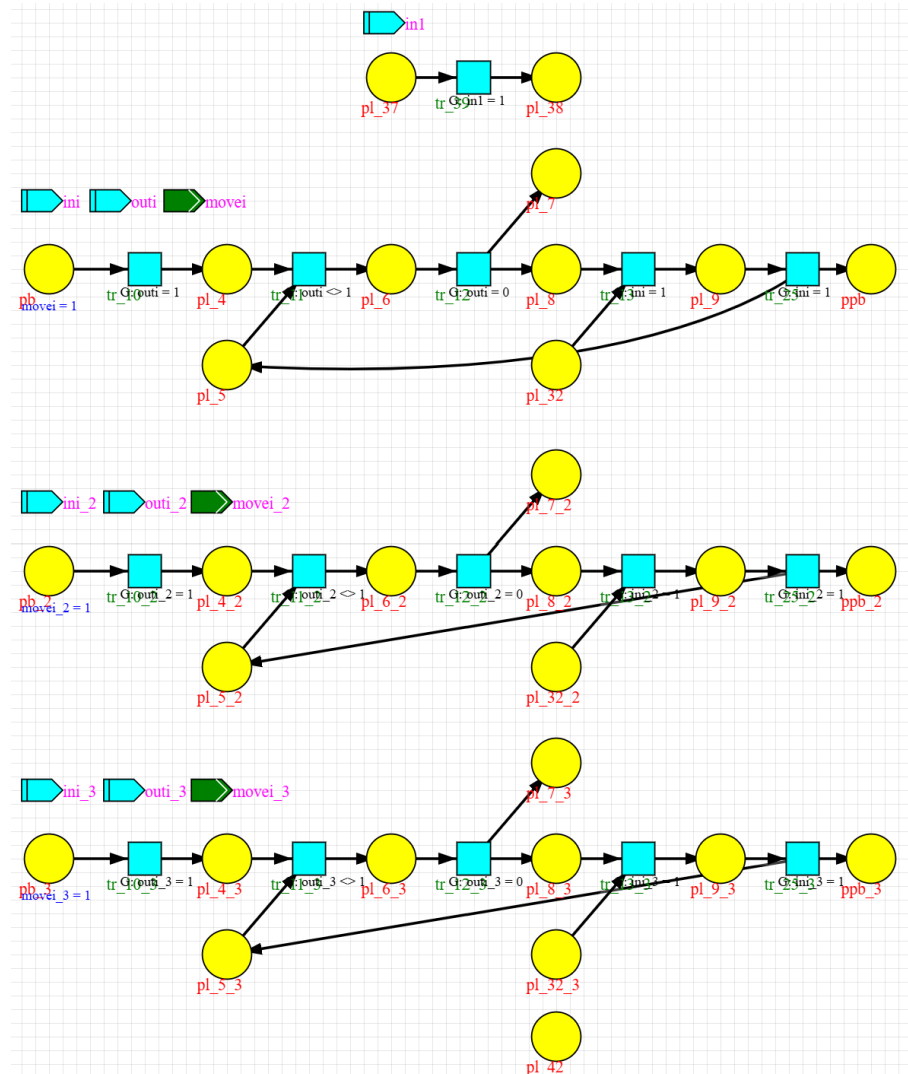


Figura 3: Rede de Petri — Modelo Completo

Seguindo os passos do problema I, realizamos a fusão dos conjuntos de lugares e transições, como se pode observar nas figuras 4.

3.3 Rede de Petri — Fusion set

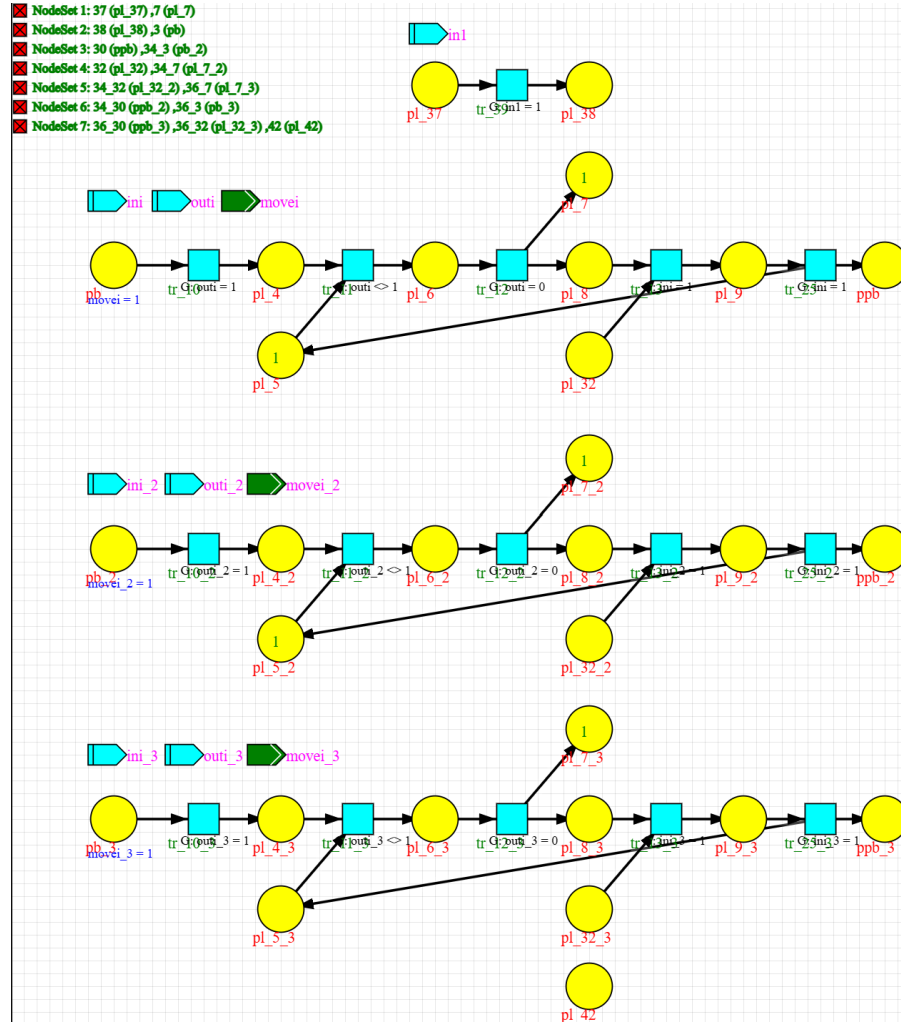


Figura 4: Rede de Petri — Fusion set

Por fim utilizando o botão "net addition" da ferramenta IOPT-Tools, obtemos o modelo final do sistema, que se pode observar na figura 5. Este sistema implementa o controlador descrito no enunciado do problema I, de forma centralizada e síncrona.

3.3.1 Rede de Petri — Node set

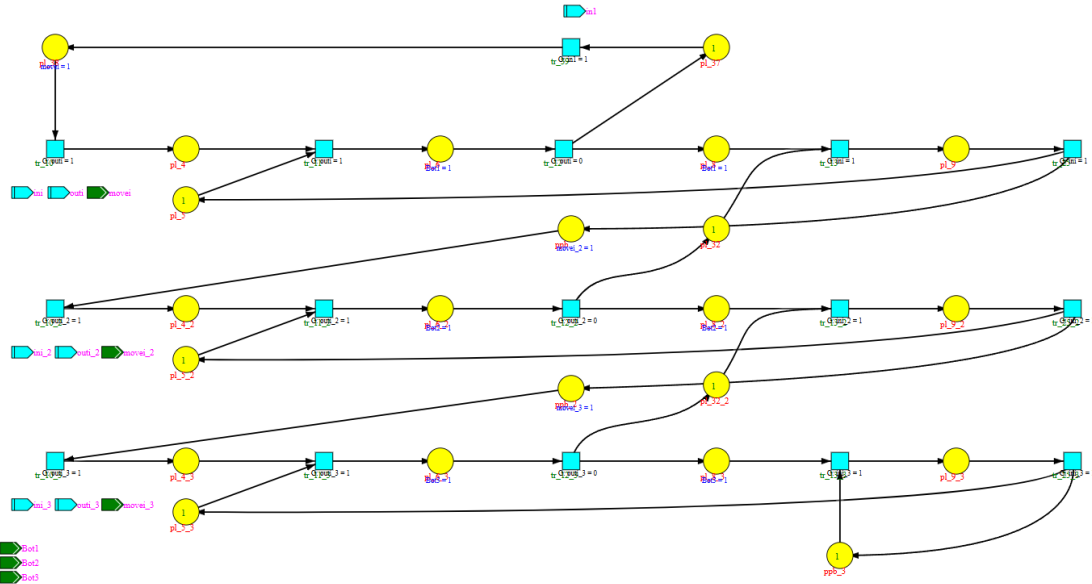


Figura 5: Rede de Petri - Node set

4 Problema II - Análise do modelo criado

Neste problema é nos pedido que validemos o modelo criado no problema I, recorrendo a simulação. e por fim é nos pedido que geremos o espaço de estados.

4.1 Simulação com token-player

Neste link tem acesso ao vídeo da simulação com token-player na plataforma IOPT-Tools do modelo "node set" .:

[Vídeo - Simulação com token-player do modelo "node set"](#)

4.1.1 Geração e análise do espaço de estados

```

586884 Started state-space generation.

Cycle 1: 1 states + 0 links
Cycle 2: 2 states + 0 links
Cycle 3: 3 states + 0 links
Cycle 4: 4 states + 0 links
Cycle 5: 5 states + 0 links
Cycle 6: 8 states + 0 links
Cycle 7: 13 states + 4 links
Cycle 8: 18 states + 10 links
Cycle 9: 25 states + 18 links
Cycle 10: 34 states + 30 links
Cycle 11: 51 states + 48 links
Cycle 12: 70 states + 90 links
Cycle 13: 103 states + 174 links
Cycle 14: 146 states + 306 links
Cycle 15: 213 states + 510 links
Cycle 16: 304 states + 808 links
Cycle 17: 351 states + 1396 links
Cycle 18: 432 states + 1852 links
Cycle 19: 474 states + 2393 links
Cycle 20: 524 states + 2769 links
Cycle 21: 566 states + 3145 links
Cycle 22: 578 states + 3373 links
Cycle 23: 598 states + 3485 links
Cycle 24: 606 states + 3601 links
Cycle 25: 614 states + 3657 links
Cycle 26: 622 states + 3713 links

MIN Bounds: p1_32=0 p1_32_2=0 p1_37=0 p1_38=0 p1_4=0 p1_4_2=0 p1_4_3=0 p1_5=0 p1_5_2=0 p1_5_3=0 p1_6=0 p1_6_2=0 p1_6_3=0 p1_8=0 p1_8_2=0 p1_8_3=0 p1_9=0 p1_9_2=0 p1_9_3=0 ppb=0 ppb_2=0 ppb_3=0
MAX Bounds: p1_32=1 p1_32_2=1 p1_37=1 p1_38=1 p1_4=1 p1_4_2=1 p1_4_3=1 p1_5=1 p1_5_2=1 p1_5_3=1 p1_6=1 p1_6_2=1 p1_6_3=1 p1_8=1 p1_8_2=1 p1_8_3=1 p1_9=1 p1_9_2=1 p1_9_3=1 ppb=1 ppb_2=1 ppb_3=1

#####
Total States: 622
Total Links: 3741
#####

Executing queries...
Done: found 0 query matching states.

Generation time (sec): 0.00 (when 0.00 it is smaller than 0.01sec)

Generating output file.
Done.

#####
Total States: 622
Total Links: 3741
Deadlock count: 0
Conflict count: 0
Invalid count: 0
#####

```

Figura 6: Rede de Petri — Espaço de Estados

Atendendo á figura 6, podemos observar o espaço de estados do modelo "node set".

- Este espaço de estados contém 622 estados possíveis com 3741 ligações entre estados.
- Não existem deadlocks, conflitos ou ligações inválidas no sistema.

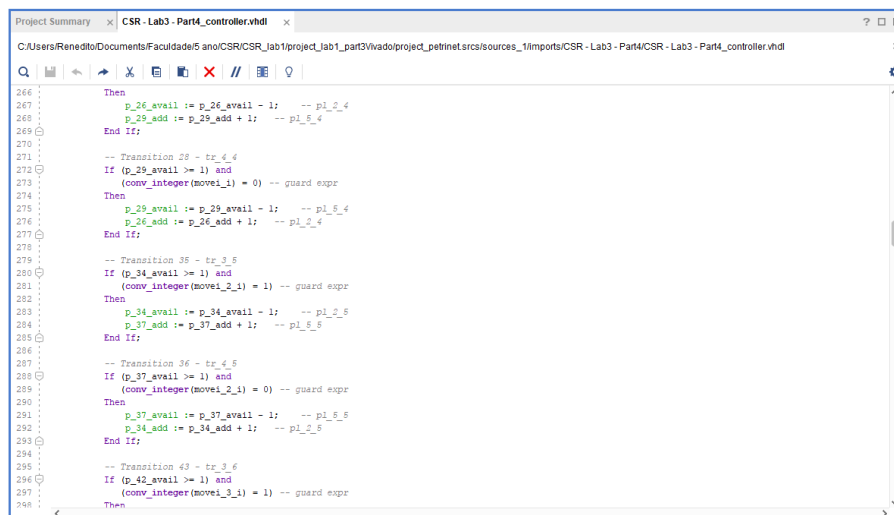
Feito isto utualizados os bounds e geramos o código VHDL.

5 Problema III - Implementação em FPGA

Nesta secção, detalha-se o processo de implementação do sistema em uma FPGA (Field-Programmable Gate Array).

5.1 Geração do código VHDL

O código VHDL encontra-se descrito na figura abaixo.



```

266 :
267 :   Then
268 :     p_26_avail := p_26_avail - 1; -- pl_2_4
269 :     p_26_add := p_26_add + 1; -- pl_5_4
270 :   End If;
271 :
272 :   -- Transition 29 - tr_4_4
273 :   If (p_29_avail >= 1) and
274 :     (conv_integer(move1_1) = 0) -- guard expr
275 :   Then
276 :     p_29_avail := p_29_avail - 1; -- pl_5_4
277 :     p_26_add := p_26_add + 1; -- pl_5_4
278 :   End If;
279 :
280 :   -- Transition 35 - tr_3_5
281 :   If (p_34_avail >= 1) and
282 :     (conv_integer(move1_2_1) = 1) -- guard expr
283 :   Then
284 :     p_34_avail := p_34_avail - 1; -- pl_2_5
285 :     p_37_add := p_37_add + 1; -- pl_5_5
286 :   End If;
287 :
288 :   -- Transition 36 - tr_4_5
289 :   If (p_37_avail >= 1) and
290 :     (conv_integer(move1_2_1) = 0) -- guard expr
291 :   Then
292 :     p_37_avail := p_37_avail - 1; -- pl_5_5
293 :     p_34_add := p_34_add + 1; -- pl_5_5
294 :   End If;
295 :
296 :   -- Transition 43 - tr_3_6
297 :   If (p_42_avail >= 1) and
298 :     (conv_integer(move1_3_1) = 1) -- guard expr
299 :   Then

```

Figura 7: Código VHDL gerado para implementação em FPGA

Este código foi gerado automaticamente a partir do modelo de Rede de Petri utilizando a framework IOPT-Tools, que traduz as especificações do modelo para uma descrição em VHDL adequada para síntese em FPGA.

5.2 Configuração de entradas/saídas

Para este projeto, foram configuradas as seguintes entradas e saídas na FPGA:

```
Port(  
  Clk : IN STD_LOGIC;  
  ini : IN STD_LOGIC;  
  outi : IN STD_LOGIC;  
  ini_2 : IN STD_LOGIC;  
  outi_2 : IN STD_LOGIC;  
  ini_3 : IN STD_LOGIC;  
  outi_3 : IN STD_LOGIC;  
  inl : IN STD_LOGIC;  
  Bot1_i : IN STD_LOGIC;  
  Bot2_i : IN STD_LOGIC;  
  Bot3_i : IN STD_LOGIC;  
  movei_i : IN STD_LOGIC;  
  movei_2_i : IN STD_LOGIC;  
  movei_3_i : IN STD_LOGIC;  
  Bot1 : OUT STD_LOGIC;  
  Bot2 : OUT STD_LOGIC;  
  Bot3 : OUT STD_LOGIC;  
  movei : OUT STD_LOGIC;  
  movei_2 : OUT STD_LOGIC;  
  movei_3 : OUT STD_LOGIC;  
  inl_o : OUT STD_LOGIC;  
  ini_o : OUT STD_LOGIC;  
  outi_o : OUT STD_LOGIC;  
  ini_2_o : OUT STD_LOGIC;  
  outi_2_o : OUT STD_LOGIC;  
  ini_3_o : OUT STD_LOGIC;  
  outi_3_o : OUT STD_LOGIC;  
  Enable : IN STD_LOGIC;  
  Reset : IN STD_LOGIC  
);  
End CSR_Lab3_Part4;
```

Figura 8: Configuração de entradas e saídas na FPGA

5.3 Simulações no Vivado

Na figura abaixo, encontra-se uma simulação realizada no ambiente Vivado, que demonstra o comportamento do sistema implementado na FPGA.

Este teste consiste em aplicar o conjuntos de sinais de entrada que permite fazer o sistema evoluir dês o estado inicial até ao estado final, onde todas as peças foram processadas corretamente pelos tapetes transportadores e braços robóticos de acordo com a seguinte sequência:

- Caixa no Tapete 1 (*saída - movi* == '1')
- Caixa no Braço 1 (*saída - bot1* == '1')
- Caixa no Tapete 2 (*saída - movi_2* == '1')
- Caixa no Braço 2 (*saída - bot2* == '1')
- Caixa no Tapete 3 (*saída - movi_3* == '1')
- Caixa no Braço 3 (*saída - bot3* == '1')

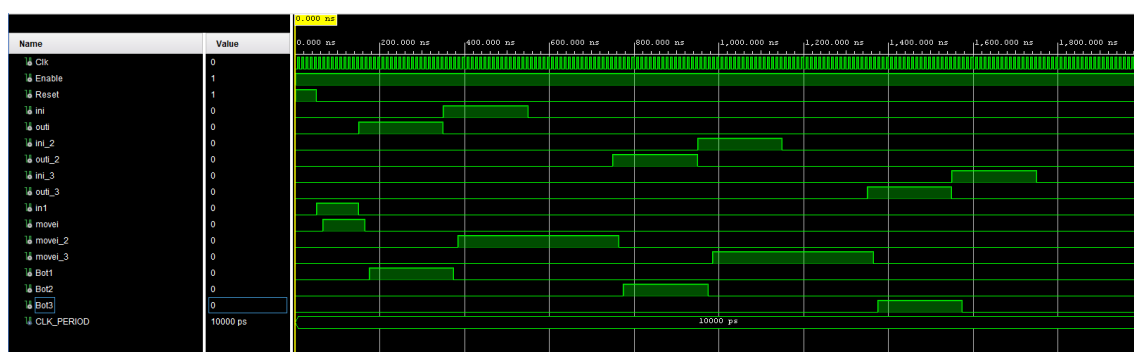


Figura 9: Simulação do sistema no Vivado

Esta simulação demonstra o correto processamento de uma caixa através dos tapetes transportadores e braços robóticos, validando a funcionalidade do sistema implementado na FPGA.

5.4 Testes Experimentais

Os testes experimentais foram realizados para validar o funcionamento do sistema implementado na FPGA. Os resultados obtidos foram comparados com os resultados da simulação para garantir a consistência e a precisão do modelo.

Neste link tem acesso ao vídeo dos testes experimentais realizados na FPGA:

[Vídeo - Testes experimentais na FPGA - Problema III](#)

Quando comparados com os resultados da simulação, os testes na FPGA demonstraram um desempenho consistente, validando a eficácia do modelo implementado.

6 Problema IV - Implementação em Arduino

Nesta seção foi nos pedido que implementássemos o sistema desenvolvido no problema I em um microcontrolador Arduino Uno.

Para isso foi necessário utilizar o dispositivo FPGA para ter acesso a um conjunto de LEDs e switches que permitem o utilizador interagir com o sistema implementado no Arduino.

6.1 Implementação no arduino e Geração do código C

O código C encontra-se descrito na figura abaixo.

```

net_main.ino  net_exec_step.c  net_functions.c  net_io.c  net_types.h
1  /* Net_CSR_Lab_3_Part1 - IOPT */
2  /* Automatic code generated by IOPT2C XSLT transformation. */
3
4
5  #include <stdlib.h>
6  #include "net_types.h"
7
8
9  int trace_control = TRACE_CONT_RUN;
10
11 extern void httpServer_init();
12 extern void httpServer_getRequest();
13 extern void httpServer_sendResponse();
14 extern void httpServer_disconnectClient();
15 extern void httpServer_checkBreakPoints();
16 extern void httpServer_finish();
17
18 static CSR_Lab_3_Part1_NetMarking marking;
19 static CSR_Lab_3_Part1_InputSignals inputs, prev_inputs;
20 static CSR_Lab_3_Part1_PlaceOutputSignals place_out;
21 static CSR_Lab_3_Part1_EventOutputSignals ev_out;
22
23 void setup()
24 {
25     createInitial_CSR_Lab_3_Part1_NetMarking( &marking );
26     init_CSR_Lab_3_Part1_OutputSignals( &place_out, &ev_out );
27     CSR_Lab_3_Part1_InitializeIO();
28     CSR_Lab_3_Part1_GetInputSignals( &prev_inputs, NULL );
29 #ifdef HTTP_SERVER
30     httpServer_init();
31 #endif
32 }
33
34 void loop()
35 {
36 #ifdef HTTP_SERVER
37     httpServer_getRequest();
38 #endif
39
40     if( trace_control != TRACE_PAUSE )
41     {
42         CSR_Lab_3_Part1_ExecutionStep( &marking, &inputs, &prev_inputs, &place_out, &ev_out );
43     }
44     else CSR_Lab_3_Part1_GetInputSignals( &inputs, NULL );
45     if( trace_control > TRACE_PAUSE ) --trace_control;
46 #endif

```

Figura 10: Código C gerado para implementação em Arduino

Este código foi gerado automaticamente a partir do modelo de Rede de Petri utilizando a framework IOPT-Tools, que traduz as especificações do modelo para uma descrição em C adequada para execução em microcontroladores Arduino.

6.2 Configuração de entradas/saídas

Para este projeto, foram configuradas as seguintes entradas e saídas no Arduino:

```

net_main.ino  net_exec_step.c  net_functions.c  net_io.c  net_types.h
49
50
51
52
53 /* Read all hardware input signals and fill data-structure */
54 void CSR_Lab_3_Part1_GetInputSignals(
55     CSR_Lab_3_Part1_InputSignals* inputs,
56     CSR_Lab_3_Part1_InputSignalEvents* events )
57 {
58     inputs->out1 = digitalRead( 31 );
59     inputs->ini = digitalRead( 30 );
60     inputs->out1_2 = digitalRead( 33 );
61     inputs->ini_2 = digitalRead( 32 );
62     inputs->out1_3 = digitalRead( 35 );
63     inputs->ini_3 = digitalRead( 34 );
64     inputs->in1 = digitalRead( 36 );
65
66 #ifdef HTTP_SERVER
67     if( input_fv != NULL ) force_CSR_Lab_3_Part1_Inputs( input_fv, inputs );
68 #endif
69 }
70
71
72 /* Write all output values to physical hardware outputs */
73 void CSR_Lab_3_Part1_PutOutputSignals(
74     CSR_Lab_3_Part1_PlaceOutputSignals* place_out,
75     CSR_Lab_3_Part1_EventOutputSignals* event_out,
76     CSR_Lab_3_Part1_OutputSignalEvents* events )
77 {
78 #ifdef HTTP_SERVER
79     if( output_fv != NULL )
80         force_CSR_Lab_3_Part1_Outputs( output_fv, place_out, event_out );
81 #endif
82     digitalWrite( 49, place_out->move1 );
83     digitalWrite( 51, place_out->move1_2 );
84     digitalWrite( 53, place_out->move1_3 );
85     digitalWrite( 48, place_out->Bot1 );
86     digitalWrite( 50, place_out->Bot2 );
87     digitalWrite( 52, place_out->Bot3 );
88 }
89
90
91
92 /* Delay between loop iterations to save CPU and power consumption */
93 void CSR_Lab_3_Part1_Delay( )

```

Figura 11: Configuração de entradas e saídas no Arduino

Do lado da FPGA foi preciso mapear os Headers JC e JD da FPGA para os pinos digitais do Arduino, conforme o port mapping presente na figura abaixo.

```

-----
-- Map JD switches/buttons to JD outputs
-----

JD1 <= in1;
JD2 <= out1;
JD3 <= ini;
JD4 <= out1_2;
JD5 <= ini_2;
JD6 <= out1_3;
JD7 <= ini_3;

-----
-- Map JC inputs (from Arduino) to LED outputs
-----

move1 <= JC1;
move1_2 <= JC2;
move1_3 <= JC3;
Bot1 <= JC4;
Bot2 <= JC5;
Bot3 <= JC6;

```

Figura 12: Mapeamento dos Headers JC e JD da FPGA

6.3 Testes Experimentais

Os resultados dos testes no Arduino foram comparados com os resultados da simulação e da implementação em FPGA.

Neste link tem acesso ao vídeo dos testes experimentais realizados no Arduino:

[Vídeo - Testes experimentais no Arduino - Parte IV](#)

A análise revelou que o desempenho do Arduino é consistente com as previsões do modelo, embora possa apresentar algumas limitações em termos de velocidade e capacidade de processamento em comparação com a FPGA.

7 Problema V - Controlador Distribuído em regime Síncrono

7.1 Cutting Set

A identificação do conjunto de corte (cutting set) é um passo crucial na decomposição de sistemas distribuídos. Este conjunto consiste em um grupo de canais ou conexões que, quando removidos, dividem o sistema em subsistemas independentes. A escolha adequada do cutting set permite a implementação eficiente de controladores distribuídos, minimizando a complexidade da comunicação entre os módulos.

7.2 Decomposição SPLIT

A decomposição usando SPLIT (Synchronous Parallel Logic Interconnect Technology) permite a criação de uma arquitetura mais flexível e escalável, dividindo o sistema em módulos independentes que se comunicam através de canais síncronos. Essa abordagem facilita a implementação de sistemas complexos, permitindo a reutilização de componentes e a redução do tempo de desenvolvimento. A decomposição SPLIT para esta aplicação pode ser visualizada na figura abaixo.

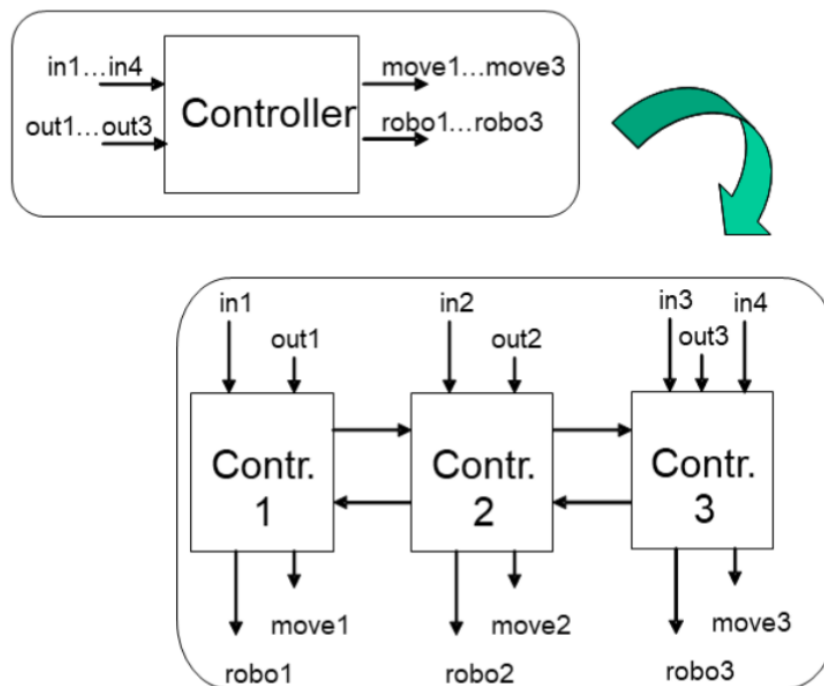


Figura 13: Decomposição SPLIT do sistema em módulos independentes.

7.3 Modelo com canais síncronos

A modelação com canais síncronos envolve a definição de canais de comunicação que operam em sincronia, permitindo a troca de informações entre os módulos de forma coordenada. Essa abordagem é fundamental para garantir a consistência e a previsibilidade no comportamento do sistema. Esta modelação encontra-se representada na figura abaixo.

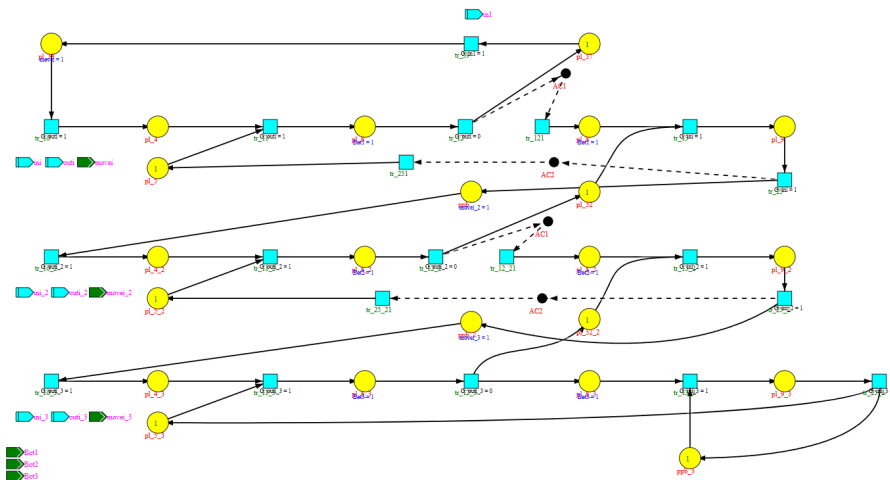


Figura 14: Modelo com canais síncronos entre os módulos.

7.4 Modelo com canais assíncronos

A modelação com canais assíncronos envolve a definição de canais de comunicação que operam de forma independente, permitindo a troca de informações entre os módulos sem a necessidade de sincronização. Essa abordagem é útil em sistemas onde a latência e a largura de banda são variáveis, proporcionando maior flexibilidade na comunicação. Este modelo está descrito na figura abaixo.

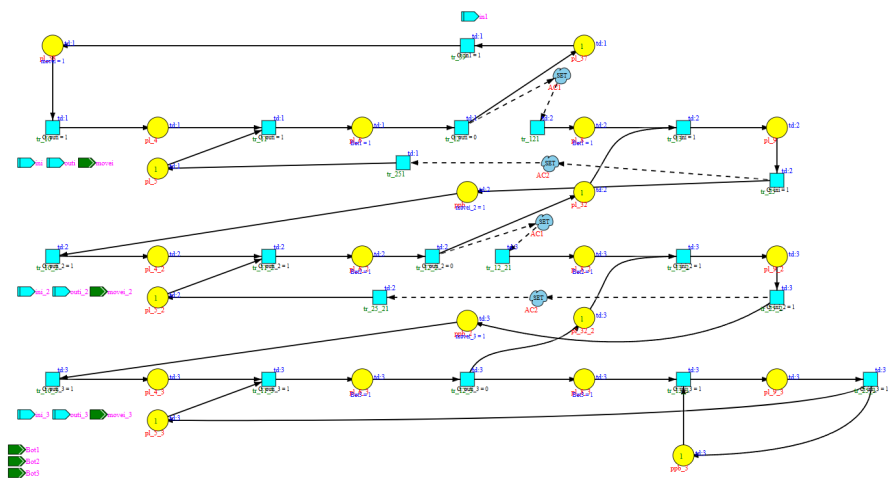


Figura 15: Modelo com canais assíncronos entre os módulos.

7.5 Decomposição GALS

A decomposição GALS (Globally Asynchronous, Locally Synchronous) envolve a criação de três controladores separados que operam de forma independente, permitindo uma maior flexibilidade e escalabilidade no sistema. Essa abordagem facilita a integração de diferentes módulos e a adaptação a variações nas condições de operação. A decomposição GALS para esta aplicação pode ser visualizada nas figuras abaixo.

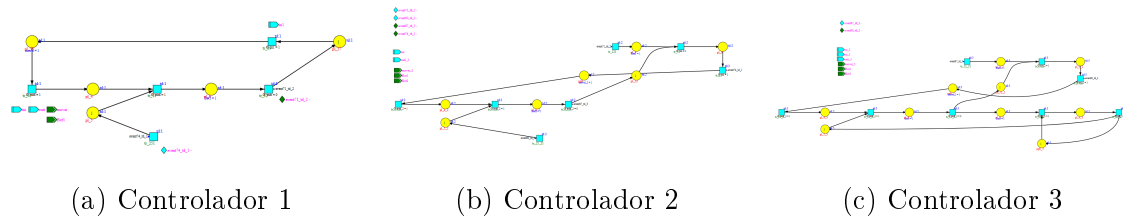


Figura 16: Decomposição GALS — Controladores 1–3 com controle por eventos

7.6 Simulação e Análise

7.6.1 Simulação com token-player

abaixo estão os links para os vídeos das simulações realizadas com o token-player para os modelos com canais síncronos e assíncronos.

- Vídeo — decomposição do modelo com canais síncronos - Problema V parte a)
- Vídeo — decomposição do modelo com canais assíncronos - Problema V parte b)

8 Problema VI - Implementação em FPGA com distribuição Síncrona

Neste problema é nos pedido á semelhança do problema III para implementar o sistema desenvolvido no dispositivo FPGA, mas desta vez com o controlador Distribuído.

8.1 Geração do código VHDL

Começamos por gerar o código VHDL do sistema a partir do modelo de Rede de Petri utilizando a framework IOPT-Tools, que traduz as especificações do modelo para uma descrição em VHDL adequada para síntese em FPGA.

Neste momento o sistema é composto por três módulos independentes que comunicam entre si através de canais síncronos, conforme a decomposição SPLIT apresentada anteriormente.

Criando um ficheiro main, foi possível ligar todos estes compoentes corretamente.

8.2 Simulação no Vivado

No ambiente do vivado, utilizamos o ficheiro de simulação desenvolvido previamente para testar o comportamento dos sistema distribuído conforme na figura abaixo:

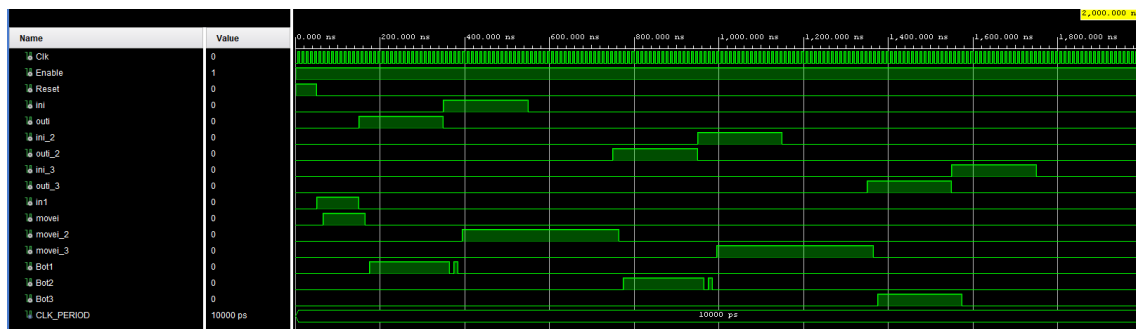


Figura 17: Simulação do sistema distribuído em FPGA com canais síncronos

Atendendo á defenição para as saídas do sistema em 5.3 podemos observar o correto funcionamento do sistema distribuído implementado na FPGA para o tratamento de uma caixa.

8.3 Testes Experimentais

Os testes experimentais foram realizados para validar o funcionamento do sistema implementado na FPGA. Os resultados obtidos foram comparados com os resultados da simulação para garantir a consistência e a precisão do modelo.

Neste link tem acesso ao vídeo dos testes experimentais realizados na FPGA:

[Vídeo - Testes experimentais na FPGA \(com LFSR\)](#)

A análise revelou que o desempenho da FPGA é consistente com as previsões do modelo, demonstrando a eficácia da implementação e a precisão do sistema projetado.

9 Problema VII - Implementação em FPGA com distribuição Síncrona com atrasos entre eventos

Tal como no problema VI, neste problema é nos pedido á semelhança do problema IV para implementar o sistema desenvolvido no dispositivo FPGA, mas desta vez com o controlador Distribuído e com a inclusão de atrasos entre eventos utilizando um LFSR.

Visto que a implementação é análoga á do problema VI, apenas serão descritos os aspetos específicos relacionados com a implementação do LFSR e a gestão dos atrasos no sistema.

9.1 Implementação do LFSR

O LFSR (Linear Feedback Shift Register) é um componente essencial para a geração de números pseudoaleatórios no sistema. A sua implementação em VHDL envolve a definição dos registos de shift (deslocamento) e das funções de feedback necessárias para garantir a pseudo-aleatoriedade dos valores gerados. Usando como referência o documento fornecido pelos docentes [1] , procedeu-se à implementação do LFSR com as características desejadas.

Abaixo está um testbench simples para validar o funcionamento do LFSR implementado, nele colocamos 3 pulsos á entrada e observamos a sua saída. No testbnch a variável `r[23:0]` é o valor presente no LFSR num dado instante, isto traduz-se no número de ciclos de clock que demorará o atraso:

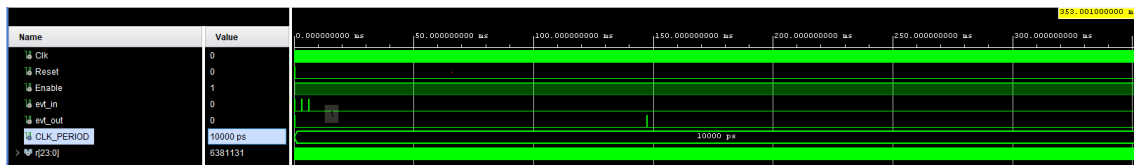


Figura 18: Simulação do LFSR implementado em VHDL

Na figura 19 Podemos ver que após ser aplicado um sinal de entrada no LFSR foi possível observar um sina de saída após aproximadamente $10\mu s$



Figura 19: Simulação do LFSR implementado em VHDL ampliado no primeiro pulso

De seguida para o segundo sinal de entrada, podemos observar na figura 20 que o sinal de saída é gerado após aproximadamente $147ms$

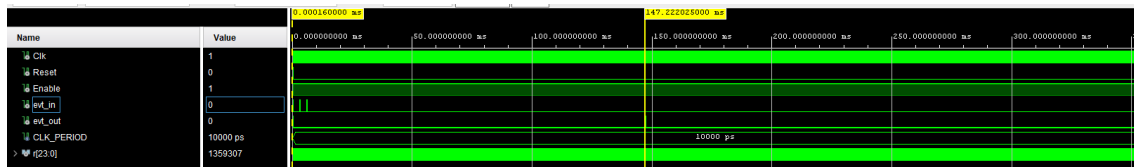


Figura 20: Simulação do LFSR implementado em VHDL ampliado no segundo pulso

No caso do ultimo sinal de entrada, este não aparece na saída do LFSR visto que o dispositivo não tem memória e este sinal foi ativado enquanto ele estava no processo de contagem.

9.1.1 Gestão dos atrasos

A gestão dos atrasos no sistema é crucial para garantir a sincronização adequada entre os diferentes componentes. No caso do LFSR, os atrasos são implementados para assegurar que os valores gerados sejam utilizados de forma eficiente e sincronizada com o restante sistema. A implementação dos atrasos é feita através de registos e sinais que controlam o fluxo de dados, garantindo que os valores gerados pelo LFSR sejam disponibilizados no momento certo para os outros módulos do sistema.

9.2 Testes Experimentais

Os testes experimentais foram realizados para validar o funcionamento do sistema implementado na FPGA. Os resultados obtidos foram comparados com os resultados da simulação para garantir a consistência e a precisão do modelo.

Neste link tem acesso ao vídeo dos testes experimentais realizados na FPGA com LFSR:

[Vídeo - Testes experimentais na FPGA \(com LFSR\) - Problema VII](#)

A análise revelou que o desempenho da FPGA é consistente com as previsões do modelo, demonstrando a eficácia da implementação e a precisão do sistema projetado.

10 Problema VIII - Buffers de Capacidade Finita

10.1 Modificação do modelo

A modificação do modelo para múltiplas peças envolve a adaptação dos componentes do sistema para lidar com a entrada e saída de várias peças simultaneamente. Isso inclui a implementação de buffers de capacidade finita que permitem o armazenamento temporário das peças, garantindo que o sistema possa operar de forma eficiente mesmo quando há variações na taxa de entrada ou saída.

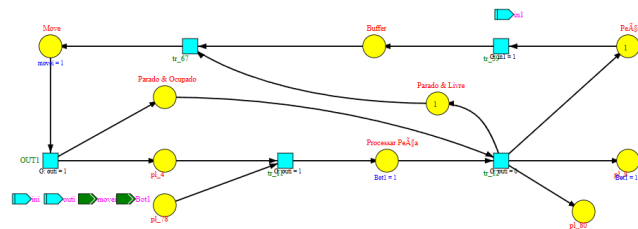


Figura 21: Rede de Petri - modelo modificado para múltiplas peças (tapete único)

10.2 Interconexão dos componentes

A interconexão dos componentes no modelo modificado é realizada através de estados capazes de gerir a presença de múltiplos objetos. Garante-se assim o fluxo das peças de uma forma mais eficiente e que os buffers possam operar corretamente dentro das suas capacidades definidas.

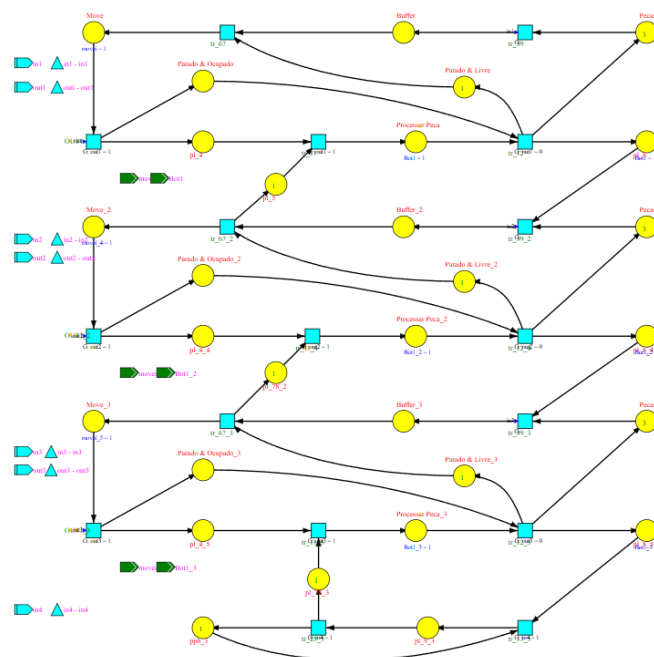


Figura 22: Rede de Petri - interconexão dos diferentes tapetes

10.2.1 Geração e análise do espaço de estados

```

1106900 Started state-space generation.

Cycle 1: 1 states + 0 links
Cycle 2: 2 states + 0 links
Cycle 3: 5 states + 0 links
Cycle 4: 10 states + 4 links
Cycle 5: 13 states + 10 links
Cycle 6: 16 states + 14 links
Cycle 7: 23 states + 20 links
Cycle 8: 40 states + 44 links
Cycle 9: 56 states + 99 links
Cycle 10: 78 states + 165 links
Cycle 11: 122 states + 279 links
Cycle 12: 200 states + 549 links
Cycle 13: 288 states + 1037 links
Cycle 14: 400 states + 1681 links
Cycle 15: 638 states + 2839 links
Cycle 16: 1100 states + 5807 links
Cycle 17: 1572 states + 10861 links
Cycle 18: 1832 states + 17669 links
Cycle 19: 2148 states + 22077 links
Cycle 20: 2600 states + 29073 links
Cycle 21: 2728 states + 35543 links
Cycle 22: 2776 states + 37599 links
Cycle 23: 2888 states + 39167 links
Cycle 24: 3000 states + 42183 links

MIN Bounds: Buffer=0 Buffer_2=0 Buffer_3=0 Move=0 Move_2=0 Move_3=0 Parado & Livre=0 Parado & Livre_2=0 Parado & Livre_3=0
Parado & Ocupado=0 Parado & Ocupado_2=0 Parado & Ocupado_3=0 Pecas=0 Pecas_2=0 Pecas_3=0 Processar Peca=0 Processar
Peca_2=0 Processar Peca_3=0 pl_4=0 pl_4_4=0 pl_4_5=0 pl_5=0 pl_78_2=0 pl_78_3=0 pl_8=0 pl_8_4=0 pl_8_5=0 pl_9_3=0 ppb_3=0

MAX Bounds: Buffer=3 Buffer_2=1 Buffer_3=1 Move=1 Move_2=1 Move_3=1 Parado & Livre=1 Parado & Livre_2=1 Parado & Livre_3=1
Parado & Ocupado=1 Parado & Ocupado_2=1 Parado & Ocupado_3=1 Pecas=3 Pecas_2=3 Pecas_3=3 Processar Peca=1 Processar
Peca_2=1 Processar Peca_3=1 pl_4=1 pl_4_4=1 pl_4_5=1 pl_5=1 pl_78_2=1 pl_78_3=1 pl_8=1 pl_8_4=1 pl_8_5=1 pl_9_3=1 ppb_3=1

#####
Total States:    3000
Total Links:     43691
#####

Executing queries...
Done: found 0 query matching states.

Generation time (sec): 0.01 (when 0.00 it is smaller than 0.01sec)

Generating output file.
Done.

#####
Total States:    3000
Total Links:     43691
Deadlock count:  0
Conflict count:  0
Invalid count:   0
#####

```

Figura 23: Espaço de estados do modelo modificado para múltiplas peças

Atendendo á figura 23, podemos observar o espaço de estados do modelo.

- Este espaço de estados contém 3000 estados possíveis com 43691 ligações entre estados.
- Não existem deadlocks, conflitos ou ligações inválidas no sistema.

10.3 Simulação e Análise

10.3.1 Simulação com token-player

Neste link tem acesso ao vídeo da simulação com token-player do modelo modificado para múltiplas peças:

[Vídeo - Simulação com token-player do modelo modificado para múltiplas peças](#)

10.4 Implementação em FPGA

Gerando novamente o código VHDL a partir do modelo modificado para múltiplas peças, foi necessário, criar um compoente relacionado com o display de 7 segmentos que nos permitir ver quantas caixas estão presentes em cada tapete durante o decorrer do programa.

A baixo vemos o testbnch realizado no Vivado:

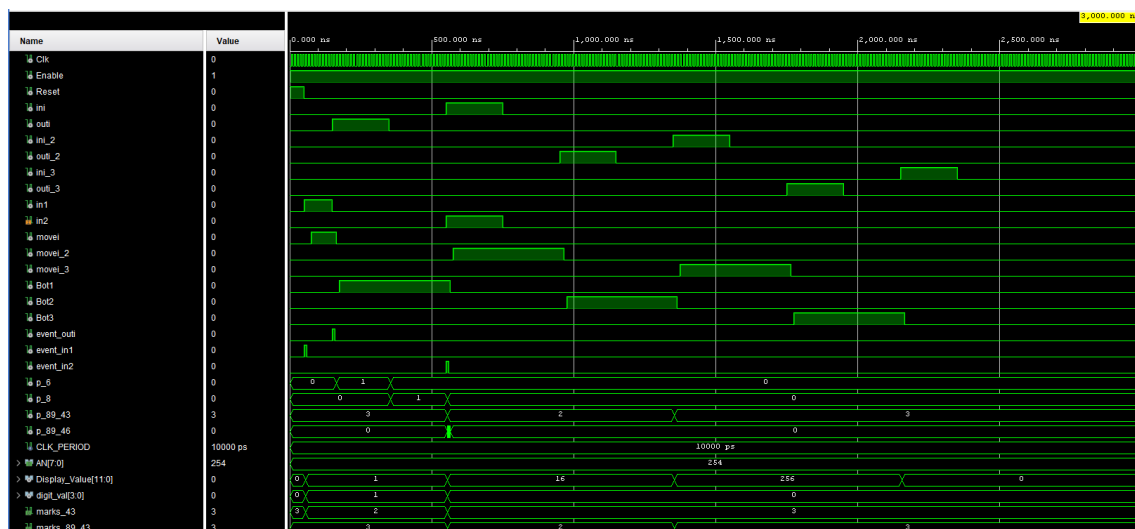


Figura 24: Testbench no Vivado - modelo modificado para múltiplas peças

A simulação confirma o correto funcionamento do sistema para lidar com uma caixa, conforme definido no modelo, com a possibilidade de visualizar as variáveis relacionadas com o display de 7 segmentos.

10.5 Testes experimentais

Neste vídeo podemos confirmar o correto funcionamento do modelo para lidar com duas caixas.

Os testes experimentais foram realizados para validar o funcionamento do sistema implementado na FPGA. Os resultados obtidos foram comparados com os resultados da simulação para garantir a consistência e a precisão do modelo.

Neste link tem acesso ao vídeo dos testes experimentais realizados na FPGA com LFSR:

[Vídeo - Testes experimentais na FPGA \(com LFSR\) - Problema VIII](#)

A análise revelou que o desempenho da FPGA é consistente com as previsões do modelo, demonstrando a eficácia da implementação e a precisão do sistema projetado.

11 Problema IX - Distribuído com Buffers (Síncrono)

11.1 Aplicação síncrona ao buffer de capacidade finita

A aplicação síncrona ao buffer de capacidade finita envolve a utilização de canais síncronos para gerir o fluxo de dados entre os diferentes módulos do sistema. Esta abordagem permite que o buffer opere de forma eficiente, garantindo que as peças sejam armazenadas e transferidas de acordo com um clock global, o que facilita a coordenação entre os componentes (minimizando a chance de perda de peças).

À semelhança do problema V efetuamos os paços necessários que levaram ao split do controlador em 3 partes:

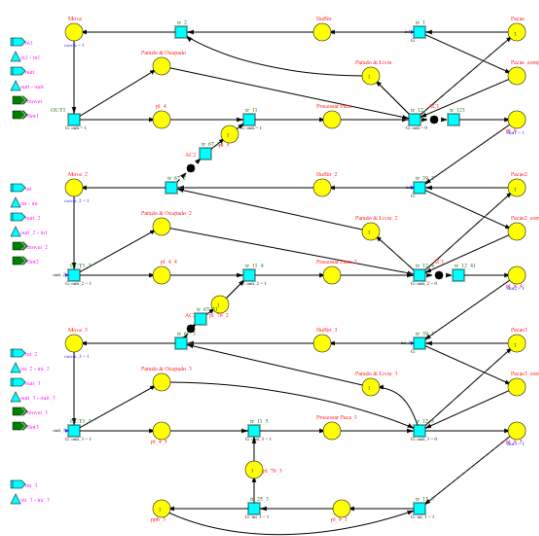


Figura 25: Decomposição do modelo com canais síncronos

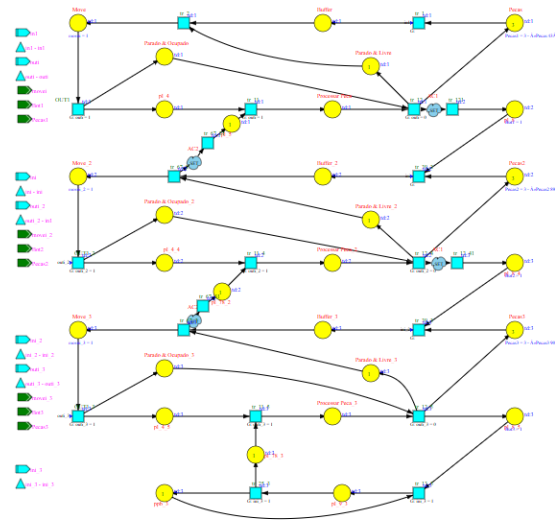


Figura 26: Decomposição do modelo com canais assíncronos

11.2 Decomposição e implementação distribuída

A decomposição e implementação distribuída do sistema com buffers de capacidade finita envolve a divisão do sistema em módulos independentes que se comunicam através de canais síncronos. Cada módulo é responsável por uma parte específica do processamento, como a gestão do buffer, a entrada e saída de peças, e o controlo dos atuadores. Esta abordagem permite uma maior flexibilidade e escalabilidade, facilitando a adaptação do sistema a diferentes requisitos operacionais.

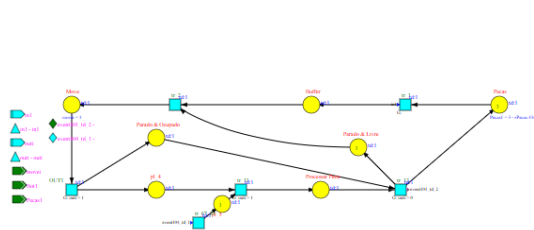


Figura 27: Rede de Petri - modelo distribuído com buffers síncronos - Controlador 1

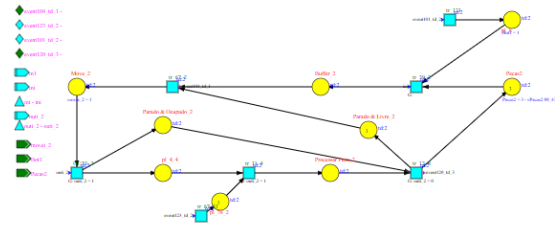


Figura 28: Rede de Petri - modelo distribuído com buffers síncronos - Controlador 2

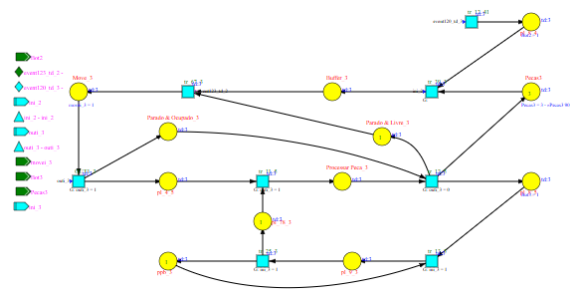


Figura 29: Rede de Petri - modelo distribuído com buffers síncronos - Controlador 3

11.3 Simulação no Vivado

Abaixo está a simulação do sistema distribuído implementado na FPGA com buffers síncronos, onde podemos observar o correto funcionamento do sistema conforme a definição para as saídas do sistema em 5.3, vemos também a variável `Display_Value[11:0]` que contem o valor do display de 7 segmentos.

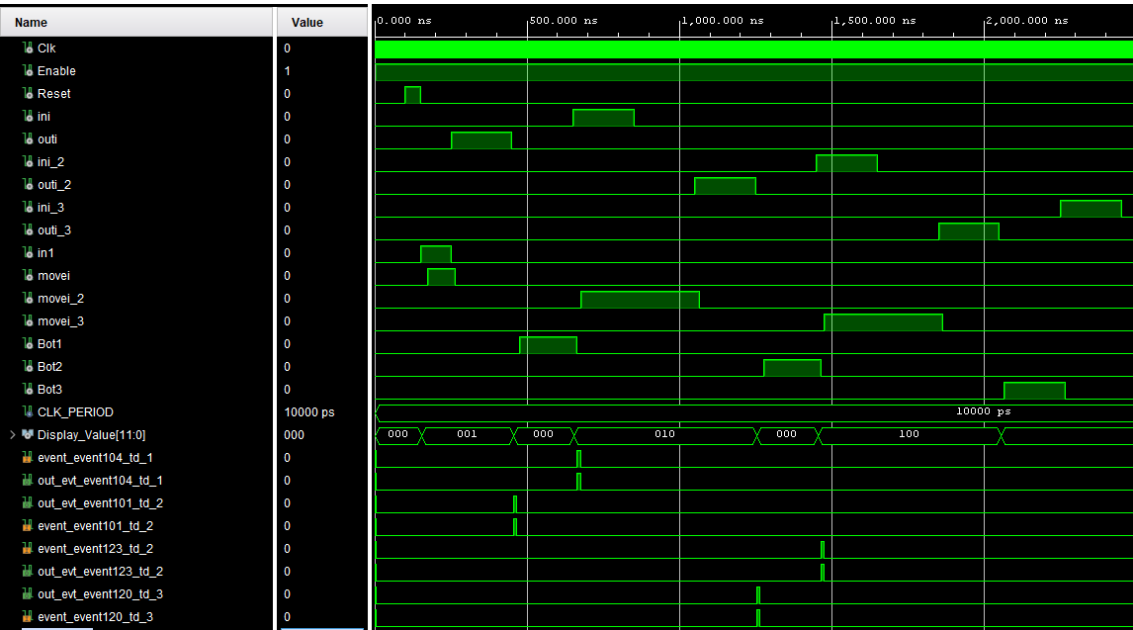


Figura 30: Simulação do sistema distribuído em FPGA com buffers síncronos

11.4 Testes Experimentais

Os testes experimentais foram realizados para validar o funcionamento do sistema implementado com buffers síncronos. Os resultados obtidos foram comparados com os resultados da simulação para garantir a consistência e a precisão do modelo.

Neste link tem acesso ao vídeo dos testes experimentais realizados com buffers síncronos:

[Vídeo - Testes experimentais dos controladores com buffers síncronos descentralizados - Problema IX](#)

A análise revelou que o sistema com buffers síncronos é capaz de operar de forma eficaz, mantendo a integridade dos dados e garantindo a sincronização adequada entre os diferentes módulos do sistema.

12 Problema X - Distribuído com Buffers (Síncrono) e atraso entre eventos

Neste problema foi nos pedido que pegássemos no sistema distribuído com buffers síncronos do problema IX e adicionássemos um atraso entre eventos, para isso foi utilizado um LFSR (Linear Feedback Shift Register) que gera números pseudoaleatórios que são usados para introduzir atrasos variáveis entre os eventos do sistema.

Reunindo tanto os controladores em VHDL como o código relacionado com o LFSR, somente foi necessário construir um ficheiro main que interligasse estes componentes para realizar esta parte do trabalho.

12.1 Simulação no Vivado

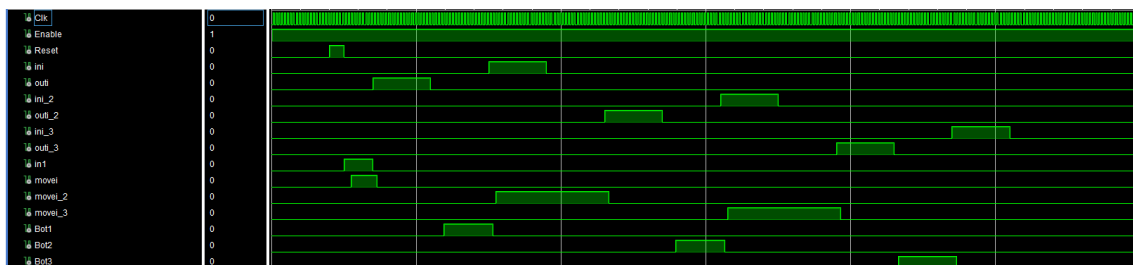


Figura 31: Simulação do sistema distribuído em FPGA com buffers assíncronos e atraso entre eventos

Nesta simulação podemos observar o correto funcionamento do sistema conforme a definição para as saídas do sistema em 5.3.

12.2 Testes Experimentais

Os testes experimentais foram realizados para validar o funcionamento do sistema implementado com buffers assíncronos com delays entre eventos. Os resultados obtidos foram comparados com os resultados da simulação para garantir a consistência e a precisão do modelo.

Neste link tem acesso ao vídeo dos testes experimentais realizados:

[Vídeo - Testes experimentais - Problema X](#)

13 Comparação de abordagens

13.1 Centralizado vs Distribuído

Na abordagem centralizada, todas as operações e decisões foram geridas por um único componente, o que simplificou o design e a implementação inicial. No entanto, esta abordagem levou a limitações de escalabilidade, especialmente em sistemas complexos. Na abordagem distribuída, o sistema foi dividido em múltiplos módulos independentes que comunicam entre si. Esta divisão permitiu uma maior flexibilidade, escalabilidade e resiliência, embora tenha introduzido desafios adicionais relacionados com a comunicação e sincronização entre os módulos.

13.2 Síncrono vs Assíncrono

Em regime síncrono, todos os componentes do sistema operaram em sincronia com um clock global, o que facilitou a coordenação e a previsibilidade do comportamento do sistema. No entanto, esta abordagem levou a ineficiências, especialmente quando há variações nas peças entre os tapetes. Já em regime assíncrono, os componentes operaram de forma independente, permitindo uma maior flexibilidade e adaptabilidade às variações no fluxo de peças. Esta abordagem, no entanto, introduziu complexidades adicionais na gestão da comunicação e na garantia da integridade dos dados transmitidos entre os módulos.

13.3 Impacto dos atrasos de comunicação

Os atrasos de comunicação tiveram um impacto significativo no desempenho do sistema, especialmente nas abordagens distribuídas. Em sistemas síncronos, os atrasos podem levar a falhas na sincronização, resultando em perda de peças ou congestionamento. Em sistemas assíncronos, os atrasos podem causar inconsistências na comunicação entre os módulos, exigindo mecanismos adicionais para garantir a integridade dos dados.

14 Conclusões

coloca a referencia [1] - Manual do LFSR utilizado no trabalho [2] - enunciado do trabalho.

Referências